

Subiectul 1.

a) def apartine (multime, liste):

multime $\rightarrow$ multimea primită

liste $\rightarrow$ listele primite

~~dict~~ dict = {}

dict : dicționarul ce îl vom returna

for elem in multime:

elementul curent din multime

idx = 0

a ră fie indexul listei

for lista in liste:

lista curentă

nr = lista.count(elem)

de câte ori apare un elem într-o listă

if nr != 0:

if elem not in dict:

dict [elem] = [(idx, nr)]

else:

dict [elem].append ((idx, nr))

idx += 1

return dict

b) perechi = [(a, b) for a in range (1, 11) for b in range (a+1, 11)]

c) funcția recursivă are complexitatea următoare relații de recurență:

$$T(1) = 1$$

$$T(n) = T\left(\frac{n}{2}\right) + \frac{n}{2}, \text{ pentru } n \geq 2$$

$$\rightarrow T \in O(n)$$

Lucianu Dorago - Adrian
grupa 131

din curs asta incheiamă că :

$$\begin{array}{l} a = 1 \\ b = 2 \\ p = 1 \end{array} \left\{ \begin{array}{l} \log_b a = 0 < p = 1 \Rightarrow \text{suntem în al treilea} \\ \text{cas al teoremei de master} \end{array} \right.$$

verificăm dacă $(\frac{1}{2}) < 1$ a.i.

$$a \cdot \left(\frac{n}{b}\right) \leq c \cdot f(n) \quad (\Rightarrow)$$

$$f\left(\frac{n}{2}\right) \leq c \cdot f(n) \quad (\Rightarrow)$$

$$\frac{n}{2} \leq c \cdot n \quad (\Rightarrow)$$

$$\frac{1}{2} \leq c \Rightarrow \text{pentru } c \in \left[\frac{1}{2}, 1\right) \text{ obținem că :}$$

$$T(n) \in O(n)$$

Curriculum Derages - Iobian
grupa 101

Subiectul 2.

```
lista = [int(x) for x in input('nr = ').split()]
# lista initiala
n = int(input('n = '))
n = len(lista) - 1
sort = sorted(lista, key = lambda x: -x if x < 0 else x)
# lista sortata dupa valorile absolute ale elementelor
for i in range(n, -1, -1):
    if n == 0:
        break
    if sort[i] < 0:
        sort[i] = -sort[i]
        n -= 1
    if n > 0:
        if n % 2 == 0:
            print(sum(sort))
            print(sort)
        else:
            sort[0] = -sort[0]
            print(sum(sort))
            print(sort)
    else:
        print(sum(sort))
        print(sort)
```

Algoritmul face parte din metoda de programare „greedy”, deoarece folosim un criteriu de selectie al elementelor

Algoritmul citeste lista initiala, dupa care o sortam dupa valoarea absoluta a sa a elementelor sale. Apoi cit

Funcționarea Diagnozei - Adunare
puncte 134

În timp $n > 0$ transformăm numerele negative în pozitive.
Dacă la final $n > 0$, dacă acesta este zero putem modifica
ca un element de n ori și nu o să fie modificată
lista; dacă n este negativ transformăm primul element
înapoi în negativ. Apoi afișăm suma și lista.

Complexitate:

citire: $O(n)$

sortare: $O(n \log_2 n)$

transformarea numerelor: $O(n)$

afișarea: $O(n)$

complexitate finală: $O(n \log_2 n)$

Lucianu Dragos - Adrian
grupa 131

Subiectul.

```
n) n = int(input('n = '))
t = int(input('t = '))
val = [0] * (n+1)
* vectorul se va conține soluțiile
nr = 0
* numărul de soluții
def bht(h):
    for i in range(1, t+1):
        val[h] = i
        suma = sum(val[1; h+1])
        * suma nr din vector dintre pozițiile 1 și h
        —> if suma <= n:
            if suma < n:
                bht(h+1)
            else:
                print(*val[1: h+1])
```

```
bht(1)
print('există', nr, 'soluții')
```

Algoritmul aparține tehnicii de programare „backtracking”, deoarece generează soluții și verificăm dacă sunt corecte.

Functia bht pune în vectorul valori pe poziția h numere între 1 și t. Apoi verificăm de suma <= n, dacă este suma este strict mai mică apelăm bht(h+1), dacă este egală opriam soluția și mărim numărul de soluții.

b) singura instrucțiune ce trebuie să fie modificată este primul if din bht cu:

```
if suma <= n and (h == 1 or val[h] < val[h-1]):
```